

Advanced Manual **Smart Contract Audit**

September 30, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

Nlyra

Real Yield Staking v2, NlyraFeeSplitter.sol

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	3	0
● Low-Risk	2	0	1	1
● Medium-Risk	0	0	0	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Nlyra

Website	nlyra.xyz
---------	-----------

Blockchain	Robinhood Chain (Arbitrum Orbit, chainId 4663)
------------	--

Source Code	github.com/NlyraProject/nlyra-platform, staking-v2/src (commit 19880c0)/NlyraFeeSplitter.sol (143 lines)
-------------	--

Contract Address	0x8300Ef5cC02cAb1D141dBE1c0B33d8Ac115F2D48
------------------	--

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Audit Summary	2
Audit Scope	3
Audit Method	3
Automated Vulnerability Check	3
Manual Code Review	3
Used tools	3
Table of Contents	4

● The Contract Has No Owner And No Settings At All

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
FSP-I1	This contract has no owner, no settings and no way for anyone to withdraw from it. The split between stakers and treasury and the minimum time between collections are fixed when it is created, and we found no hidden path to redirect the money or pull funds out. The trade off is that nothing can be repaired either: if the fee source ever changes its behaviour, the only remedy is to redirect fees at the source and replace this contract entirely.

Code Snippet

```
// no owner, no setters, no withdrawal; everything fixed at creation
uint256 public immutable SPLIT_BPS;
uint256 public immutable MIN_HARVEST_INTERVAL;
address public immutable STAKING;
address public immutable TREASURY;

// the only exit is at the source, outside this contract:
// the fee owner redirects with setFeeRedirect
```

Recommendation

No change is needed. Make sure the escape route is written down and that the team knows who holds the ability to redirect fees at the source, since that is the only lever available if this contract ever needs to be replaced.

● The Argument That Self Donating For The Bonus Cannot Be Profitable Checks Out

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
FSP-I2	Money routed through this contract counts towards the five percent compounding bonus paid by the staking contract, which raises the obvious question of whether someone could send funds through it to farm that bonus. The staker share is capped at 90 percent specifically to prevent this, and we checked the arithmetic: even for someone who owned the entire staking pool the round trip returns about 94.5 percent of what they put in, so it always loses money.

Code Snippet

```
uint256 public constant MAX_SPLIT_BPS = 9_000; // at most 90% to stakers

// worst case for the protocol, an attacker owning the whole pool:
//   0.90 reaches stakers, bonus multiplies it by 1.05
//   0.90 * 1.05 = 0.945 returned for 1.00 spent
// so the round trip always loses money
```

Recommendation

No change is needed. If the split is ever reconsidered, keep the 90 percent ceiling, because the protection depends on the treasury share always being larger than the bonus.

● Two Small Points On Validation And Housekeeping

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
FSP-I3	Two small points. When this contract is created it checks that the fee source is a real contract but does not run the same check on the token, staking or treasury addresses; in practice the staking contract creates this one and passes in addresses it has already validated, so the gap is closed by how it is deployed rather than by the contract itself. Separately, the stray token sweep does not require a non zero balance, so it can emit a record saying a sweep happened when nothing actually moved.

Code Snippet

```
// only the fee source is checked for being a real contract
if (locker.code.length == 0) revert NotAContract();
// nlyra, weth, staking and treasury are not code checked here

// sweep does not require a non zero balance
uint256 bal = IERC20(token).balanceOf(address(this));
IERC20(token).safeTransfer(TREASURY, bal);
emit Swept(token, bal); // fires even when bal is 0
```

Recommendation

Keep deploying this contract from the staking contract, which is what makes the missing checks harmless, and note that requirement wherever the deployment is documented. Adding a check that there is actually a balance before sweeping would keep the event log honest, at no real cost.

● Anyone Can Trigger A Collection, And Doing So Always Resets The Waiting Period

Low-Risk - Won't cause any problems, but can be adjusted for improvement [Status: Resolved]

Error Code	Description
FSP-L1	Anyone can trigger a collection, which sensibly means the protocol never depends on one party remembering to do it, but the waiting period resets on every successful call including one that collects almost nothing, so an outsider can trigger a collection at the least useful moment and delay the next meaningful distribution. How long that delay can be is set by a value fixed when the contract is created. We confirmed from the deployment script that it is set to one day, the shortest the contract permits, that it is asserted again after deployment, and that it cannot be changed afterwards, so for this deployment the worst case is a single day rather than the seven the contract would otherwise allow. Fees keep accruing at the source and are collected in full next time.

Code Snippet

```
// fixed at creation and not changeable afterwards
uint256 public immutable MIN_HARVEST_INTERVAL;

// deployment script, and re-checked after deploying:
uint256 constant MIN_HARVEST_INTERVAL = 1 days;    // 86,400 s
require(... sp.MIN_HARVEST_INTERVAL() == MIN_HARVEST_INTERVAL, "locker");

// so the worst case delay here is 1 day, not the allowed maximum of 7
```

Recommendation

No further action needed for this deployment, since the interval is fixed at one day and cannot be raised. If these contracts are ever redeployed, keep the interval at the short end of the allowed range for the same reason.

● The Bonus Tag On A Handover Can Be Dropped Without Anyone Noticing

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
FSP-L2	When handing money to the staking contract, this contract also reports how much of it came from protocol fees, because only that portion earns the compounding bonus. If the amount is below the staking contract's minimum, that contract keeps the money but opens nothing and the note about where it came from is not stored, so the money later counts as an ordinary reward. Neither contract reports that the tag was dropped, and the sums involved are small by definition.

Code Snippet

```
// the message is sent unconditionally, with no result checked
IRealYieldStakingNotify(STAKING).notifyRewards(ws, ns);

// on the staking side, below the minimum nothing is opened
// and the eligible portion is not carried forward
```

Recommendation

Only trigger a handover once the amounts are comfortably above the staking contract's minimum, so the eligible portion is never handed over in a form that cannot be recorded. Alternatively, have the staking contract carry the untagged eligible amount forward, which is the more complete fix and is recommended in the staking report as well.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.